

📞 연락처

[이메일]

[전화번호]

[GitHub]

⚡ 핵심 역량

- LLM 파인튜닝 및 RAG 시스템 설계
- FastAPI 기반 AI 플랫폼 백엔드 구축
- LangGraph 에이전트 설계 및 최적화
- pgvector 기반 벡터 검색 시스템 구현

🔧 기술 스택

Hub-Spoke MCP 아키텍처 설계

AI/ML

ExaOne 3.5, LLaMA, Gemini, BGE-m3, LangChain, LangGraph, PEFT, TRL, PyTorch

백엔드

FastAPI, Uvicorn, Pydantic, SQLAlchemy, Alembic

DB/벡터

PostgreSQL, pgvector(HNSW), FAISS

프론트

Next.js 16, React 19, TypeScript, Tailwind CSS, Zustand

인프라

tmdb, Workpod(PWA), Alembic 마이그레이션 [자격증 1]

[자격증 2]

🌐 언어 능력

한국어 (모국어)

영어 ([수준])

[이름]

AI 풀스택 개발자 / LLM 플랫폼 엔지니어

"LLM·RAG·에이전트를 활용한 AI 인사·성과 플랫폼을 설계·구현한 개발자"

🎓 학력

[대학교] [학과] [졸업년도]

👛 프로젝트 경력

RAG — AI 인사·성과 플랫폼 (개인 프로젝트, 2024~2025)

- 역량·공시 데이터와 LLM(ExaOne 3.5)을 결합한 AI 인사·성과 플랫폼 설계·구현
- LangGraph 기반 RAG 에이전트 구축: disclosures·competency_anchors·employees·performance_records 4개 테이블 벡터 검색
- FastMCP Hub-Spoke 아키텍처 설계: Chat MCP(ExaOne)·Spam MCP(LLaMA) 분리 운영
- LLaMA 스팸 분류기 파인튜닝: 한국어 15종 스팸 + 햄 SFT 데이터 생성·학습 (검증 정확도 88%+)
- BGE-m3 임베딩 + pgvector HNSW 인덱스 기반 고속 벡터 검색 구현
- ExaOne·Embedding Lazy 로딩으로 t3.small 인스턴스에서 OOM 없이 운영
- 채팅 SSE 스트리밍, 임베딩 배치 처리(32건), 이력서 세션 캐시로 응답 성능 개선
- Alembic 마이그레이션 021 리비전 관리, 자동 마이그레이션 (AUTO_MIGRATE)
- Store-then-Process 수신 메일 아키텍처 + Mailgun Webhook 연동 구현
- Next.js 16 + React 19 프론트엔드 구현: 대시보드·채팅·직원관리·성과·감사·데이터지도·워크스페이스 등 15개+ 화면
- 이력서 → LLM 분석 → Success DNA(5대 역량 0~100점) 자동 산출 기능
- 성화 지표 (구현 기준)
- 전체 API 라우터 8종 (직원·성과·채팅·공시·이력서·감사·문서·메일) 완성
- 프론트엔드 15개+ 화면 구현 및 백엔드 연동
- LLM 1회 호출 최적화 (기존 2회 → 1회, GPU 부하 50% 감소)
- 스팸 분류 정확도 88%+ 달성 (3,600건 SFT 학습 기준)

RAG 플랫폼 — 기술 구조 & 구현 상세

AI 기반 인사·성과·공시 플랫폼 (개인 프로젝트 / 전체 구현 완료)

시스템 아키텍처



주요 구현 모듈



RAG 채팅 에이전트

- LangGraph: rag → model → tool → stream
- 1턴 최적화: `_build_forced_tool_calls` 적용
- 4테이블 벡터 검색 + 하이브리드(tsvector)
- OOS 처리, 명단 30명 상한, GPU 메모리 즉시 해제



메일·스팸 시스템

- Store-then-Process: 수신 즉시 저장, 비동기 워커
- LLaMA 스팸 분류: 15종 한국어 SFT (정확도 88%+)
- inbox 저장 시 성과/역량 자동 분류 연동
- Mailgun Webhook(HMAC), ai_status 상태 관리



직원·Success DNA

- Success DNA: 이력서 LLM 분석 (5대 역량 점수화)
- BGE-m3 임베딩 일괄 갱신 및 pgvector 검색
- API 페이지네이션 적용으로 로딩 최적화
- 이력서 파일 해시 중복 방지 및 프로필 백필



성과·감사·공시

- 성과: 활동 내역 CRUD, 업무 제출, 직원별 조회
- 감사: 직원 생성/수정/삭제 전 과정 로그 자동 기록
- 공시: 문서 텍스트 추출, 임베딩 구축, RAG 검색
- 공시 기여도 예측 동기/비동기 API 지원

성능·최적화 요약

항목	구현 방식	최적화 효과
LLM 호출 최적화	1턴 구조, <code>forced_tool_calls</code> 활용해 2회 → 1회 단축	GPU 부하 50% 감소 및 OOM 방지
스트리밍 응답	LangGraph SSE 적용, 실시간 청크 전송	첫 토큰 대기시간 대폭 단축, 체감 속도 향상
임베딩 배치 처리	32건/512건 단위의 <code>embed_documents(batch)</code> 적용	API 호출 오버헤드 감소, 대량 처리 효율화
Lazy 로딩	서버 기동 시 DB만 확인, 모델/인덱스는 첫 요청 시 로드	t3.small 등 소형 인스턴스 OOM 방지
이력서 세션 캐시	동일 파일 재업로드 시 프론트엔드 캐시에서 즉시 반환	불필요한 LLM 재분석 방지 (로딩 0초)